

Experiment 3

Name: N. PARNITA

Course Code: MLA0305

Regno: 192525322

Slot: B

Title: Policy and Value Function Representation using Bellman Equations

Software Required:

- Python
- Jupyter Notebook
- NumPy
- Matplotlib

AIM

To implement the Bellman Equation in Python and represent the Value Function for different states in a Markov Decision Process (MDP).

THEORY

The Bellman Equation is used in Reinforcement Learning to calculate the value of a state. It tells us the expected future reward obtained from a state by considering the immediate reward and discounted future rewards.

Bellman Equation:

$$V(s) = R(s) + \gamma V(s')$$

where

- $V(s)$ = Value of current state
- $R(s)$ = Immediate reward
- γ (**gamma**) = Discount factor
- $V(s')$ = Value of next state

PROCEDURE

1. Open Jupyter Notebook or Google Colab.
2. Import NumPy and Matplotlib libraries.
3. Define the states of the MDP.
4. Assign rewards for each state.
5. Set the discount factor (γ).
6. Initialize the value function.
7. Apply the Bellman Equation iteratively.
8. Display the final value of each state.
9. Plot the value function using Matplotlib.
10. Observe the convergence of values.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

# States
states = ['S1', 'S2', 'S3', 'Terminal']

# Rewards
rewards = [5, 2, 10, 0]

# Discount factor
gamma = 0.9

# Initialize Value Function
V = np.zeros(len(states))

# Bellman Iteration
for iteration in range(20):
    new_V = V.copy()

    for i in range(len(states)-1):
        new_V[i] = rewards[i] + gamma * V[i+1]

    V = new_V
```

```
print("Final Value Function")

for s, value in zip(states, V):
    print(s, ":", round(value,2))

# Plot
plt.figure(figsize=(6,4))
plt.bar(states, V)
plt.title("State Value Function using Bellman Equation")
plt.xlabel("States")
plt.ylabel("Value")
plt.grid(axis='y')
plt.show()
```

OUTPUT:

Final Value Function

S1 : 14.9

S2 : 11.0

S3 : 10.0

Terminal : 0.0

